MATERIAL DE CURSO

Trilha: comece a programar com IA sem afundar o projeto

PADRÃO DE ENGENHARIA UNIVERSAL

Guia de Uso · versão 3.2

Como construir sistemas, apps, SaaS e sites com qualidade profissional
— com ou sem inteligência artificial no teclado.

Constituição · Handbook · Kit de Partida · Auditoria automática · Pontos de restauração

O que você vai encontrar

O QUE É

Quatro arquivos que funcionam juntos como um sistema operacional de engenharia: norma, explicação, modelos e auditoria automática.

PARA QUEM

Quem constrói com ajuda de IA e precisa de um resultado profissional — legível, seguro e auditável por outra pessoa meses depois.

COMO SE USA

Cole o prompt de partida, deixe o agente preparar a base do projeto e construa em fases curtas, salvando o progresso a cada uma.

Pré-requisitos: nenhum. um editor, um terminal e a disposição de não repetir o mesmo erro duas vezes.

Todo o material é gratuito, versionado e pode ser usado em projetos comerciais. Novo na v3.2: pontos de restauração — desfazer nunca mais depende da memória do chat.

Licença CC BY-ND 4.0

0 Como usar este guia

leia na ordem; cada módulo leva de três a seis minutos

Este guia acompanha quatro arquivos públicos que funcionam juntos como um **sistema operacional de engenharia**. Ele nasceu de um cenário novo: hoje boa parte do código é escrita por agentes de inteligência artificial — mas o resultado precisa continuar profissional o bastante para que qualquer desenvolvedor humano audite, entenda e mantenha. O padrão existe para garantir as duas coisas ao mesmo tempo.

Módulo	O que você aprende	Pág.
1 · Por que projetos afundam	Os seis erros que derrubam projetos feitos com IA — e o custo de cada um	2
2 · Os quatro artefatos	O que é a Constituição, o Handbook, o Kit de Partida e a auditoria	3
3 · Baixando o material	Clone, ZIP e o cuidado com versões	4
4 · Projeto novo	O fluxo completo: porte, prompt de partida, instanciação e fases	5
5 · Retomando o projeto	Como continuar de onde parou, mesmo com o contexto zerado	6
6 · Pontos de restauração	Novo na v3.2: como desfazer qualquer erro sem depender da memória da IA	7
7 · Auditoria automática	Medindo a qualidade do projeto com um comando	8
8 · As dez regras de ouro	O resumo de bolso, para colar na parede	9
9 · Licença e comunidade	O que você pode e o que não pode fazer com este material	10

1 Por que projetos afundam

os seis erros mais caros de quem programa com IA

Programar conversando com uma IA é rápido e prazeroso — até a terceira semana. E o naufrágio quase sempre vem dos mesmos seis lugares. Reconhecê-los agora custa muito menos do que descobri-los com um cliente esperando do outro lado.

ERRO QUE AFUNDA PROJETOS

1. A memória do projeto mora no chat

A janela de contexto acaba, a conversa é resumida, e as decisões combinadas somem. Semanas depois o agente "melhora" um trecho que era uma correção deliberada — e o bug volta. **Cura:** a memória mora no repositório, em arquivos criados **antes** da primeira linha de código.

ERRO QUE AFUNDA PROJETOS

2. Começar pela tela

A interface fica pronta primeiro, o modelo de dados muda depois, e todo o front é refeito. **Cura:** a ordem de construção é dados, segurança, domínio, contrato de API e só então a tela. Constrói-se do que é caro de mudar para o que é barato.

ERRO QUE AFUNDA PROJETOS

3. "Segurança a gente vê depois"

Depois nunca chega, e o sistema estreia com rotas abertas e senhas mal guardadas. **Cura:** segurança não é uma fase, é transversal — nasce junto com a primeira rota e acompanha cada entrega.

ERRO QUE AFUNDA PROJETOS

4. Confiar no "testei e funciona"

O agente afirma que verificou, mas não verificou. É a falha número um de quem programa com IA. **Cura:** toda afirmação de teste vem com a evidência colada — a saída real do comando. Sem evidência, não aconteceu.

ERRO QUE AFUNDA PROJETOS

5. Instalar qualquer biblioteca da moda

Pacotes com nome parecido com o famoso, ou versões recém-publicadas com código malicioso, entram no projeto junto com trojan e ransomware. **Cura:** quarentena — versão publicada há menos de duas semanas não entra (exceto correção de segurança), somente registro oficial, e scripts de instalação desligados por padrão.

ERRO QUE AFUNDA PROJETOS · NOVO NA v3.2

6. A IA erra, apaga o que funcionava — e não consegue desfazer

Uma edição ruim destrói código que estava certo; você pede para desfazer, e o agente responde que "não tem mais acesso à memória do que fez". É verdade: a memória dele é volátil. **Cura:** pontos de restauração — commit obrigatório **antes** de qualquer operação de risco, e um diário que registra o que cada ponto contém. Desfazer vira um comando do git, não uma lembrança da IA.

2 Os quatro artefatos

cada um tem um leitor e uma função diferente

Arquivo	Função	Quem lê
CONSTITUIÇÃO	**A norma.** Cerca de 150 regras com identificador estável, níveis de exigência e parâmetros de contexto que adaptam o padrão ao seu projeto.	A IA (colada na sessão)
HANDBOOK	**O porquê.** Vinte e quatro capítulos explicando cada regra, quando ela não se aplica e os erros mais comuns de quem tenta segui-la.	Você (consulta humana)
KIT DE PARTIDA	**O executável.** Modelos prontos dos arquivos de memória — incluindo o diário de pontos de restauração — e três textos para colar: iniciar projeto, retomar projeto e gerar a versão enxuta da norma.	Você e a IA
audit.sh	**O fiscal.** Script que verifica sozinho as regras automatizáveis em qualquer projeto e devolve um relatório com aprovações e violações.	O terminal

Os quatro princípios que sustentam tudo

Princípio	Tradução prática
Memória no repositório	Nada que importa vive apenas na conversa. Decisão tomada no chat vira arquivo antes do fim da sessão.
Ordem de construção	Dados, depois segurança, depois domínio, depois a API e por fim a tela. A segurança acompanha tudo.
Evidência sempre	Nenhuma entrega é dada como pronta sem a saída do teste, do build ou da requisição colada junto.
Desfazer é git, não memória	O git guarda o conteúdo de cada ponto; o diário de restauração guarda o significado. Reverter um erro nunca depende da lembrança da IA.

3 Baixando o material

clone, ZIP e o cuidado com versões

Opção A — clonar com git (recomendado: mantém histórico e facilita atualizações):

```
git clone https://github.com/Viny-Atriaon/padrao-engenharia-universal-publico.git
cd padrao-engenharia-universal-publico
```

Opção B — baixar o ZIP: abra o repositório no navegador, clique no botão verde **Code** e depois em **Download ZIP**. Extraia numa pasta sua.

Trabalhe sempre sobre uma versão com etiqueta. Cada publicação estável recebe uma tag (por exemplo, v3.2). Assim você sabe exatamente qual texto o seu projeto seguiu, mesmo que o padrão evolua depois:

```
git tag             # lista as versoes publicadas
git checkout v3.2   # trava o material na versao 3.2
```

NA PRÁTICA

Este repositório é somente leitura

Você pode clonar, baixar, usar em projetos pessoais e comerciais e compartilhar o link. Ninguém além do mantenedor consegue alterar ou apagar o conteúdo publicado — o que garante que a versão citada hoje continuará existindo amanhã.

4 Começando um projeto novo

do zero até a primeira entrega, na ordem certa

Passo 1 — Escolha o porte

O porte define quanto rigor o projeto exige. Ele nunca dispensa a memória (os arquivos de contexto); apenas ajusta as camadas técnicas.

Porte	Exemplos típicos	Preparação
site estático	Landing page, portfólio, site institucional	Kit mínimo cerca de 10 minutos
app simples	Aplicação com backend leve, formulários, cadastro pequeno	Kit padrão cerca de 30 minutos
sistema	Sistema completo com banco de dados, login e deploy próprio	Kit completo cerca de 2 horas
SaaS	Produto com assinatura, planos e vários clientes na mesma base	Kit completo e camada SaaS

Passo 2 — Cole o prompt de partida na sessão da IA

Abra o **Kit de Partida**, seção 7, copie o texto e preencha o nome e o porte do projeto. Esse prompt obriga o agente a, **antes de escrever qualquer código**:

- executar a lista de preparação do seu porte;
- criar os arquivos de memória a partir dos modelos prontos — incluindo o diário de pontos de restauração;
- propor o plano em fases e esperar a sua aprovação;
- só então iniciar a fase 1, fechando com um ponto de salvamento.

Passo 3 — Gere a versão enxuta da norma

Colar 150 regras em toda sessão desperdiça contexto e dilui a atenção do agente: regra que não se aplica ao seu caso atrapalha as que se aplicam. Use o prompt da seção 9 do Kit para gerar um arquivo apenas com **as regras do seu contexto**. Refaça esse arquivo se o projeto mudar de porte ou de tecnologia.

Passo 4 — Construa em fatias verticais

Cada fase deve caber em uma sessão e terminar com algo demonstrável. O plano já nasce na ordem correta: preparação, modelo de dados, fundação de segurança e, a partir daí, uma fatia completa do produto por vez — sempre atravessando dados, serviço, API e tela do mesmo fluxo.

> REGRA DE OURO
>
> **Ponto de salvamento a cada fase**
>
> Fase concluída significa commit feito, arquivo de estado atualizado e uma entrada nova no diário de restauração — tudo no mesmo momento. Se perceber que a conversa está ficando longa, ou que a IA começou a resumir o histórico, faça o ponto de salvamento imediatamente — é o equivalente a salvar o jogo antes da fase difícil.

5 Retomando o projeto

a prova real de que a memória funciona

Outro dia, outra sessão, talvez outra ferramenta de IA, contexto completamente zerado. Cole o **prompt de retomada** (Kit, seção 7). O agente lê os dois arquivos de memória e responde em três linhas: onde o trabalho parou, o que vem agora e se existe algum impedimento. Depois continua do ponto exato.

> NA PRÁTICA
>
> **O teste que reprova o seu arquivo de estado**
>
> Se o agente precisar perguntar **“onde paramos?”**, o arquivo falhou. Ele deve conter sempre: a fase atual, o último ponto de salvamento, as decisões recentes, os próximos três passos concretos e os impedimentos. Nada além disso — o histórico antigo vai para outro arquivo, para que o texto continue curto e útil.

6 Pontos de restauração

novo na v3.2 — desfazer qualquer erro sem depender da memória da IA

A divisão de papéis é o coração do mecanismo: **o git é o cofre** — cada commit já guarda o projeto inteiro, byte a byte, de graça; **o diário `docs/PONTOS-DE-RESTAURACAO.md` é o mapa** — uma entrada curta por ponto de salvamento dizendo o que aquele ponto é: o que foi adicionado, modificado, corrigido ou removido, e o código (hash) do commit que o guarda.

- **Antes de toda operação de risco, a IA salva primeiro.** Edição em vários arquivos, refatoração, migração, remoção — commit antes, sempre. Se der errado, voltar é um comando.
- **O diário só cresce no fim** (nunca é reescrito), não aceita código colado (o commit já o guarda) e a entrada nasce no momento do salvamento — por isso o custo de contexto é praticamente zero.
- **Visão macro e micro de graça:** os títulos das entradas, lidos em sequência, são a linha do tempo do projeto por funcionalidade; o corpo de cada entrada detalha o que mudou item a item.

Exemplo de entrada no diário:

```
## PR-014 — 17/08/2026 — Construtor de forms: validação por campo
- Commit: `e4f5a6b` · Fase: F6 · Tipo: adicionado
- Funcionalidades tocadas: construtor-form, dashboard
- Adicionado: validação por campo (obrigatório, e-mail, máscara)
- Modificado: painel de propriedades exibe erros em tempo real
- Removido: —
- Estado verificado neste ponto: build ok, smoke ok
```

REGRA DE OURO

“Desfaça” nunca depende de lembrança

Quando algo der errado, a ordem para o agente é uma só: restaurar a partir do hash registrado no diário (`git restore --source <hash>` ou `git revert`). A IA re-digitar “de cabeça” o que apagou é proibido — foi exatamente assim que projetos se perderam.

7 Auditoria automática

medindo a qualidade com um comando

O script confere as regras que uma máquina consegue verificar e imprime um relatório. Funciona em Windows (pelo Git Bash), Linux e macOS:

```
bash audit.sh /caminho/do/seu/projeto
bash audit.sh /caminho/do/seu/projeto --deps     # inclui auditoria de dependencias
```

Exemplo de relatório real:

```
============================================================
 AUDITORIA - Constituicao Universal v3.2 - meu-projeto
============================================================
 [OK] GIT-INIT-01            repositorio git presente
 [OK] SECRETS-GITIGNORE-01   .env nao rastreado
 [X]  CTX-ESTADO-01          sem arquivo de estado - memoria viva ausente
 [OK] CTX-RESTORE-01         diario de pontos de restauracao presente
 [OK] PIPE-SUPPLY-01a        lockfile presente (pnpm-lock.yaml)
 [!]  DEP-QUARENTENA-01      scripts de instalacao nao restringidos
============================================================
 RESULTADO: 14 conformes  |  1 violacao  |  7 alertas
```

Violações viram pendências registradas no arquivo de instruções do projeto, com prazo. Rode a auditoria a cada marco importante e sempre antes de entregar para um cliente.

8 As dez regras de ouro

o resumo de bolso do padrão inteiro

Regra	O que significa na prática
Preparação antes do código	Nenhuma linha de código antes de os arquivos de memória existirem no repositório.
Ponto de salvamento	Fase concluída é igual a commit mais arquivo de estado atualizado. Conversa ficando longa? Salve agora.
Desfazer é git, não memória	Antes de operação de risco, commit primeiro; cada ponto de salvamento entra no diário de restauração com o hash. Reverter usa o git — nunca a lembrança da IA.
Decisão vira arquivo	O que foi combinado no chat é gravado em decisão registrada ou no arquivo de estado antes de a sessão acabar — senão, não existe.
Ordem de construção	Dados, segurança, domínio, API e por fim a tela. A segurança acompanha tudo e nunca fica para depois.
Escala em duas moedas	O que é barato (aplicação sem estado, paginação, índices) entra no primeiro dia; o que é caro (balanceador, réplicas) só com dor medida.
Documentação junto da entrega	Rota criada é rota documentada no mesmo ponto de salvamento. Deixar para documentar no final é proibido.
Evidência ou não aconteceu	“Testei e funciona” exige a saída do teste colada. Sem ela, a verificação não ocorreu.
Quarentena de dependências	Versão publicada há menos de duas semanas não entra, exceto correção de segurança. Somente registro oficial.
Código para humanos	Teste dos dez minutos: alguém que nunca viu o projeto entende qualquer módulo apenas com o código e o README.

REGRA DE OURO

Se você só levar uma frase deste guia

O chat é memória volátil; o repositório é memória permanente. Tudo que importa — decisão, estado, correção, ponto de restauração, documentação — precisa estar em arquivo versionado. É isso que separa um projeto que sobrevive à troca de sessão, de ferramenta e de pessoa, de um projeto que afunda na primeira semana difícil.

9 Licença e comunidade

o que você pode e o que não pode

Este material está sob a licença **Creative Commons BY-ND 4.0** (atribuição, sem derivações).

Você PODE	Você NÃO PODE
Usar em qualquer projeto — pessoal, comercial ou de clientes — sem pedir permissão.	Publicar versões modificadas do padrão como se fossem o material original.
Clonar, baixar e compartilhar os arquivos originais, dando crédito ao repositório.	Remover a atribuição ao autor e ao repositório de origem.
Gerar a versão instanciada do seu projeto — preencher parâmetros é uso normal, não modificação.	Alterar ou apagar o repositório oficial: é tecnicamente impossível, por ser somente leitura.
Adaptar internamente para a sua equipe, desde que a adaptação não seja redistribuída.	Aplicar restrições extras que impeçam outras pessoas de usar o material.

Contribuindo

Sugestões, correções e relatos de uso são bem-vindos: abra uma **Issue** no repositório. O padrão evolui por prática real, nunca por acúmulo de teoria — toda regra nova precisa ter sido aplicada com sucesso em pelo menos um projeto de verdade antes de entrar (foi assim que os pontos de restauração da v3.2 nasceram: de um incidente real). Mudanças aceitas viram uma versão nova, com etiqueta própria.

Próximo passo

Escolha um projeto pequeno de verdade — uma página, um formulário, um cadastro — e percorra o fluxo inteiro uma vez: porte, prompt de partida, fases, ponto de salvamento e auditoria. O padrão só vira hábito depois da primeira volta completa.